

Spatial-Temporal Multi-Tasking Model

A Major Qualifying Project

Submitted to the Faculty of

WORCESTER POLYTECHNIC INSTITUTE (WPI)

In partial fulfillment of the requirements for the

degree of Bachelor of Science

By Nicholas Ishigami, William Abraham, and Jeffrey Chan

25 March 2025

Report submitted to:

Professor Yanhua Li, WPI

This report represents the work of one or more WPI undergraduate students submitted to the faculty as evidence of completion of a degree requirement. WPI routinely publishes these reports on the web without editorial or peer review.

Contents

1 – Introduction	4
2 – Preliminaries	5
2.1 - Large Foundation Models	6
2.1.1 - Pretraining	6
2.1.2 - Fine-Tuning Model	7
2.1.3 - Downstream Tasks	8
2.1.4 - Masking	9
2.2 – Commonly Used Data	10
2.2.1 - Spatial-Temporal Data	11
2.2.2 - Sequential Data	12
3 – Methodology	12
3.1 – Data	13
3.1.1 - Explored Data Sources	13
3.1.2 - Data Description	14
3.1.3 - Data Cleaning and Preparation	15
3.1.4 - Data Masking Strategy	16
3.2 - Model Implementation	17
3.2.1 – Pretraining	18
3.2.2 - Downstream Tasks	19
3.2.2.1 – Trip Duration Model	20
3.2.2.2 – Pickup Zone Model	20
3.2.2.3 – Dropoff Zone Model	21
3.2.3 - Baseline Models	21
4 – Results	23
4.1 – Pretraining Results	23
4.1.1 - Masking All Numerical Features	24
4.1.2 - Masking Numerical and Categorical Features	26
4.1.3 - Masking Subsets of Features	28
4.2 – Baseline Model Results	29

4.2.1 - Trip Duration Model	30
4.2.2 – Pickup Zone Model	31
4.2.3 – Dropoff Zone Model.....	34
4.3 - Fine tuning Results	36
4.3.1 - Trip Duration Model	37
4.3.2 – Pickup Zone Model	38
4.3.3 – Dropoff Zone Model.....	41
5 – Conclusions.....	43
6 - Appendix.....	44
Appendix I.....	44
Appendix II	44
Appendix III	45
Appendix IV	46
Appendix V	47
Appendix VI.....	47
7 - References.....	49

Abstract

In today's world, large-scale models are widely used for various tasks, yet there remains a gap in models specifically designed for spatial-temporal data. Such data is crucial for applications like urban traffic management and optimizing delivery routes. Given the success of generative models in language processing, advancing spatial-temporal models is a natural progression in AI development. This paper presents our approach to building a spatial-temporal model, the challenges encountered, and experimental results demonstrating its feasibility and importance.

1 – Introduction

Spatial-temporal data, which captures both space and time aspects of real-world events and movement, is increasingly important in a wide range of fields including urban planning, traffic prediction, meteorology, epidemiology, and more [1][2]. With the widespread use of GPS-enabled devices, IoT (Internet of Things) sensors, and other collection methods, massive volumes of human-generated spatial-temporal data (HSTD) are now easily accessible, presenting novel opportunities and challenges for machine learning models [1][2]. Effectively making use of these datasets requires large models capable of understanding the intricate relationships between space and time to enable more accurate predictions and insights.

The recent rise of large foundation models in natural language processing (NLP), through models like BERT, GPT, and T-LNG have demonstrated the power of pretraining on vast amounts of data before fine-tuning for specific tasks [3][4][5]. This has led to significant performance improvements across various domains. However, spatial-temporal data presents unique challenges due to its sequential nature and long, complex dependencies. As such,

traditional machine learning and data processing techniques need to be adapted to handle this data [6].

In this paper, we aim to investigate the effectiveness of pretraining a large transformer-based model using spatial-temporal data and demonstrate its advantages when fine-tuning smaller models for specific downstream tasks. Our approach involves masking parts of the data during pretraining and training the model to recover missing information, allowing it to learn meaningful patterns within the data. The remainder of this report is structured as follows: Section 2 provides preliminary background information on foundation models, pretraining strategies, and the types of data we investigated. Section 3 details our methodology, including data preprocessing, model architecture, and training strategies. Section 4 presents the results of our pretraining experiments and fine-tuning on downstream tasks. Finally, Section 5 summarizes our findings and discusses future work.

2 – Preliminaries

Though in recent years the use of LLMs has become more and more apparent, there has been a lack of spatial-temporal models to analyze vast amounts of information and help with many downstream tasks. Our group aims to create a model that proves that this niche can be filled by creating a foundational model based on spatial-temporal data. In our preliminary research, we introduce the concept of foundation models, discuss various aspects of machine learning models, and review data types that are applicable to a spatial-temporal model.

2.1 - Large Foundation Models

The model we designed during our project was a type of foundation model, a model that uses a large amount of data to be applicable to a wide variety of tasks, without much task-specific labeled data. Foundation models are typically very expensive and difficult to train, as the models require large amounts of data and time training. An example of a foundation model is the DeepSeek-Coder, a model developed on two trillion tokens while also providing for both commercial and non-commercial use. Newer models like DeepSeek are already surpassing models like GPT-3.5 and Codex [7]. The purpose of one of these well-trained foundation models is to be reusable and helpful in improving model performance.

Foundational models are also currently being developed in medicine, with generalist medical AI (GMAI). These models will be capable of carrying out various tasks with little to no data, being built through self-supervision [8]. This type of foundation model will be indispensable in medical fields where low sample sizes for datasets are the norm. Recently, foundation models have also been hypothesized to have the potential to be equally as adept to tool usage as humans [9]. With the help of these foundation models, we should be able to increase accuracy, efficiency, and automation in problem solving.

2.1.1 - Pretraining

When creating a machine learning model, one of the most critical steps is pretraining your model. Pretraining refers to the task of training your foundational model on a large dataset, before using the foundational model to fine-tune a fine-tuning model. Large amounts of data are typically used for pretraining, as when the amount of augmented data (modified copies of existing data) is increased, the overall accuracy of pretraining decreases greatly [10]. If you find

yourself with a small amount of data when pretraining a model, other alternatives may be self-training or creating a fine-tuning model.

Pretraining also benefits a machine learning model by usually increasing robustness and uncertainty estimates. By pretraining a model, convergence on tasks with large datasets will speed up [11]. Foundation models also particularly rely on pretraining, as foundation models cover a large range of downstream tasks and want to accommodate a number of various fine-tuning models. Since the goal for pretraining is to learn generic patterns from the pretraining dataset, the foundation model benefits perfectly from the pretraining step.

2.1.2 - Fine-Tuning Model

In machine learning, when designing a model for a specific task, designing a fine-tuning model that utilizes a foundation model would likely be the right choice. A fine-tuning model is, put simply, a machine learning model that uses a small data source and is designed for a specific task. Fine-tuning models are very common, as many tasks do not require large models, or do not have the amount of data required to support such a model. In these cases, when a user creates a fine-tuning model, it is highly beneficial to use a foundation model to boost the fine-tuning model's pre-training accuracy.

One important note is that fine-tuning models do not have to be simple. In fact, one report that goes into depth about simpler machine learning models states that it is often much more difficult to find a simple model with good accuracy than a complex model with the same accuracy, contrary to what seems intuitive [12]. The purpose of a fine-tuning model is then to be usable while having a limited data source; or to produce reliable results relating to its specific task. In biomedical engineering, for example, small experimental groups are constantly created

for the purpose of testing new breakthroughs [13]. In these cases, machine learning models are forced to be accurate with a low amount of information, which merits creating a fine-tuning model with assistance from a foundation model.

2.1.3 - Downstream Tasks

When designing and pre-training a model, another step that machine learning experts need to consider is the creation of downstream tasks. A downstream task is simply a task that the model should be expected to perform, and one that can be used to measure the model's effectiveness in what it set out to accomplish. Typically, downstream tasks relate to the use case for a fine-tuning model; in other words, the downstream task is the specific problem that is solved by the fine-tuning model. When creating a foundation model, many downstream tasks are created as training for the model, to ensure increased testing accuracy and usability of the model.

One important step to consider when deciding on what downstream tasks to use is the relationship between pretraining and fine tuning. Typically, the "pre-train, fine-tune" paradigm suffers from the problem of inconsistent objectives, and causes suboptimal performance [14]. For this reason, downstream tasks should be planned along with the pretraining work as to not weigh too heavily on either the pre-training or fine-tuning aspects of making a machine learning model. Once downstream tasks are decided on, data collection is also important to consider for each downstream task. When deciding how much data to collect, it is better to overestimate the amount required, as according to a report from 2022, underestimating data collection for each step in the data collection process can lead to a dataset about 90% of the size required for downstream tasks [15].

2.1.4 - Masking

One very common technique for training models is the masking of information in the given dataset. Masking refers to any techniques that may obscure or hide parts of the dataset in order to better train the machine learning model. One such technique is called “Data Shuffling”, which is a hybrid procedure where the original dataset is randomly shuffled than reverse-mapped onto the original values. The desirability of this approach lies in the fact that while you are shuffling your data and disorienting the model, the marginal distribution of the new data ends up identical to the marginal distribution of the old data [16]. This approach also leads to minimal relationship loss, meaning that the model should still be able to classify the information well. Finally, by shuffling the data in this way, the values of any confidential observation are simply reassigned to another observation, meaning that the dataset is still viable for training and can be used to test the model’s accuracy. According to a paper published in 2006 discussing the methods of Data Shuffling [17], the procedure can be described as follows:

Step 1: Generate an observation y_i^p from the conditional distribution $f_{x|s}(X | S = s_i)$ such that $S = S_i$, Y^p is independent of X .

Step 2: Repeat the process for every observation in the data set to obtain Y^p .

Step 3: Replace $y_{(i),j}^p$ with $x_{(i),j}$, $(i) = 1, \dots, N$, $j = 1, \dots, M$ to obtain Y . That is, perform reverse mapping $Y^j = Y^{pj} \leftarrow X^j$.

Step 4: Release the reordered (shuffled) dataset.

The procedure of data shuffling is very important in concealing personal information in relation to ethical concerns over the use of personal data. While useful for many datasets, our dataset did not contain any non-anomized information and therefore data shuffling was not required.

Another masking technique, used on extensive datasets to save costs and time on labeling and to benefit various tasks, is the practice of simply hiding parts of each data point and having the model recover these hidden parts of information. This method of data masking, commonly referred to as masking and recovery, is used in self-supervised pre-training in order to allow for large amounts of data to be processed without scrupulous oversight. This type of masking was used for models like BERT: Google’s deep learning large model, developed in 2018. In this model, the pre-training tasks caused words to be replaced with one of three things, randomly [3]:

1 – 80% of the time, replace the word with the [Mask] token, e.g., “my dog is hairy” →

“my dog is [Mask]”

2 – 10% of the time, replace the word with a random word, e.g., “my dog is hairy” →

“my dog is an apple”

3 – 10% of the time, keep the word unchanged, to bias the representation towards the actual observed word.

This approach allows the model to learn based on any random number of variables, in our model’s case, by requiring the model to recover many different parts of data from the dataset. By randomly masking variables, you allow for a greater adaptability for the model over large generalized applications.

2.2 – Commonly Used Data

When considering a data source, it is important to keep in mind the usability and reliability of the data source. Ensuring that the data gathered is of good quality makes sure that the resulting model will be accurate and able to assist any downstream tasks with good returns, while bad data quality will hinder efforts to ensure reliable results when using the model. In this

way, it is important to choose quality data sources when selecting the data you want to train your model on. Some factors to consider when choosing data sources include the variables the dataset is made up of, the amount of data entries, the reliability of the source, and the completeness of the data. Each of these factors should be considered equally when searching for or evaluating data sources for a machine learning model.

2.2.1 - Spatial-Temporal Data

Spatial-temporal data is data that has a space (spatial) and time (temporal) component. The use for this data is in its ability to relate to trends, for both a given region and a specific datetime, and to be able to combine those two strengths into one relationship. This type of data usually contains a variable containing the date and/or time that the data entry was taken during, the location (either latitude/longitude or another specific system) where the data was captured, and some other piece of information for each data point. Some applications of this type of data would be tracking vehicles as they move around a city or capturing weather patterns in an area.

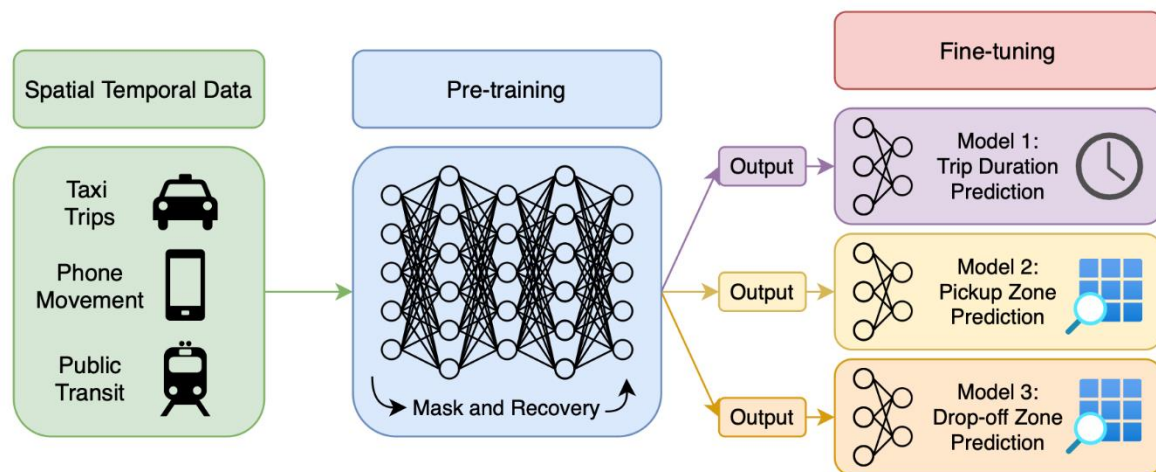
In a conference paper discussing the prediction of urban anomalies from spatial-temporal data, researchers used a three-dimensional tensor to represent anomalies in all regions of a city [18]. These dimensions were the region the anomaly took place, the category of the anomaly, and the time the anomaly took place. Given the history of a given city, they tried to predict anomalies in the future for each given area. Spatial-temporal data was crucial in making sure that these predictions were tied to real locations and were usable. Spatial-temporal data can be applied to any number of problems and datasets and is becoming more prevalent in machine learning.

2.2.2 - Sequential Data

When considering data types that may be used in a fine-tuning model, sequential data is one of the most likely types to be utilized. Sequential data refers to data that is ordered and must be considered in a particular order to be viewed correctly. This data type can be related to most data types, for example: timelines, driving routes, or weather patterns. In a spatial-temporal model, this data type is particularly important because of the sequential nature of temporal data.

According to a review of sequential data, this data type is important because sequential data can be exploited to improve the prediction accuracy for classifiers [19]. By training a model on a specific sequential dataset, the model's accuracy would be able to increase given that certain sequences are more common than others, leading to a higher accuracy when predicting information that is masked or that is not given. This also means that gathering sequential data for a fine-tuning model is important, since it is very effective for finding patterns that are reliable.

3 – Methodology



The objective of this project is to pretrain a large transformer-based model using spatial-temporal data, demonstrating the advantages of pretraining when fine-tuning smaller models for specific tasks. To achieve this, we first explored multiple data sources to identify the most suitable dataset. We then trained a transformer encoder model using a mask-and-recover strategy to learn meaningful feature representations. Finally, we developed several baseline models for downstream tasks, leveraging the pretrained model to assess its impact on performance.

3.1 – Data

Training our large model required a significant amount of data. One challenge we faced was the lack of comprehensive datasets that tracked the trajectories of all vehicles within a given timeframe. Most vehicles do not record and publish their movement data, making it difficult to obtain a complete picture of travel history. At the same time, a dataset with every vehicle’s trajectory was not strictly necessary to model traffic congestion effectively. Even a partial dataset could provide valuable insights, as traffic trends are reflected in the time it takes to complete trips as well. For example, commuting the same distance from home to work can take significantly longer during rush hour compared to late at night. This meant our focus was on finding publicly available datasets with many data points rather than complete records of every vehicle’s movement.

3.1.1 - Explored Data Sources

We explored multiple datasets that could help model traffic patterns. One option was from the California Department of Transportation (DOT), which included traffic-related datasets with estimates of total miles traveled on select public roads [20]. However, this dataset only

provided aggregate data and did not contain individual trip details, making it unsuitable for our model.

Another dataset came from Rutgers University, which had publicly available trajectory data from Shenzhen, China [21]. This dataset included GPS records from cell phones, taxis, buses, and trucks over a single day. While it offered complete vehicle trajectories, the limitation was its short duration—data from just one day would not allow the model to learn long-term patterns.

We also considered data sourced directly from the New York DOT, which was similar to California’s but included a broader catalog of publicly available, more granular datasets [22]. The New York government allows users to request additional data beyond what is posted online, but due to dataset size and travel constraints, obtaining the full data directly was challenging.

Finally, we found a publicly available dataset from a previous requester of New York government data [23]. This dataset contained New York City taxi trip records from 2013, covering an entire year. It included pickup and drop-off locations, timestamps, and other trip details, making it well suited for our needs. The dataset's extensive coverage allowed us to analyze traffic patterns over different times of the day, days of the week, and times of the year, providing valuable insights for our model.

3.1.2 - Data Description

As described in the previous section, the dataset selected for pre-training our model consists of New York City taxi trip records from 2013. The data is divided into twelve parts, one for each month, with each file containing approximately 14 million taxi trips. Each row represents a single trip and includes fourteen features that provide critical details for our model.

The dataset contains several categorical features, medallion, hack license, vendor ID, rate code, and store and forward flag. The medallion represents a transferable permit allowing a vehicle to operate as a taxi, while the hack license is a unique identifier for the driver, specific to NYC’s yellow taxi system. The vendor ID indicates the Taxicab Technology Service Provider responsible for recording the trip, and the rate code specifies the fare structure, distinguishing between standard fares, negotiated rates, group rides, and other categories. Additionally, the store and forward flag indicates whether the trip record was temporarily stored in the taxi’s memory before being transmitted to a server if it did not have a connection during the trip.

Beyond categorical attributes, the dataset includes key numerical features that describe each trip in the spatial and temporal domain. These include timestamps for pickup and drop-off times, pickup and drop-off latitude and longitude to track trip origins and destinations, trip duration in seconds, trip distance in miles, and passenger count, which records the number of riders in the taxi. The combination of these features allowed for an in-depth analysis of traffic patterns and other transportation changes throughout the year.

3.1.3 - Data Cleaning and Preparation

Although the taxi trip dataset was well-prepared and maintained, we conducted additional cleaning and preprocessing to ensure its suitability for our model. To remove extreme outliers, we first found the latitude and longitude of New York City on Google Maps, which was approximately 40 and -74 respectively [24]. Then we filtered trips based on pickup and drop-off coordinates, restricting latitude values to between 30 and 50 and longitude values to between -85 and -65. This ensured that all data points remained within the greater NYC region, eliminating erroneous records that could distort the model’s learning. Additionally, we refined the dataset’s temporal features by replacing raw pickup and drop-off timestamps with a more structured set of

time components. Each timestamp was decomposed into six separate features: year, month, day, hour, minute, and second. This transformation allowed the model to capture patterns related to time more effectively. To further enhance the data spatially, we implemented a grid system over the dataset's geographic range. By dividing the city into predefined zones, we assigned each trip a pickup and drop-off zone, enabling the model to better understand location-based traffic trends and patterns. Using our filtered dataset as described before, both the latitude and longitude were divided into segments of 0.01 degrees, or about one kilometer, starting at the minimum value of each. Each longitude was given a number corresponding to its segment index and each latitude was given a letter combination corresponding to its segment index. For example, a zone in the 105th longitude segment and 63rd latitude segment would be labeled as zone BK105. These preprocessing steps ensured that the dataset was both clean and optimized for predictive modeling.

3.1.4 - Data Masking Strategy

To effectively pretrain our model, we explored multiple data masking strategies, iterating through different approaches to determine the most effective method. Some of these initial strategies include randomly masking a single feature per trip, masking each feature independently with a set probability, and masking both numerical and categorical features simultaneously. Our final masking strategy, and the one used to train our model, involved masking numerical features by a set probability, with the same process applied to categorical features with a different set probability.

Prior to feeding the data into our model, data is passed to a masking function which selects the features to mask within each batch of data using a random number generator. Numerical features had a 1/17 chance of being masked (17 numerical features), and categorical

features had a 1/7 chance of being masked (7 categorical features). Through our testing, these probabilities made the pretraining task difficult enough to allow the model to learn patterns in the data, without making it excessively difficult. During masking, numerical features are replaced by a temporary float(nan) value, whereas categorical features are replaced by a temporary -1 value. These placeholder values are processed by the NYCFeatureEncoder (detailed in Section 3.2.1) before being fed into the model. By implementing this structured masking approach, we created a robust pretraining task that enhances the model’s ability to recover missing information and learn relationships in the dataset.

3.2 - Model Implementation

In this section, we detail the implementation of our model, including the pretraining task and its application to downstream tasks. Our approach follows a two-stage process: first, we pretrain a transformer-based model on using a mask-and-recovery task to learn meaningful representations of taxi trip data. Then, we evaluate the effectiveness of this pretrained model by applying it to three downstream tasks—predicting trip duration, pickup zone, and drop-off zone.

Pretraining serves as the foundation of our approach, allowing the model to capture spatial and temporal dependencies in the taxi data before being fine-tuned for specific predictive tasks. By masking and recovering key features, we encourage the model to understand patterns in the data, such as the relationship between time, location, and other trip details.

The downstream tasks were carefully selected to assess different aspects of the model's learning. Predicting trip duration tests the model's ability in understanding time-based relationships, while pickup and drop-off zone predictions evaluate its spatial understanding.

These tasks align with real-world applications in transportation analytics, where accurate predictions can enhance traffic forecasting.

3.2.1 – Pretraining

The core of our pretraining approach is the TaxiMaskRecovery model, which consists of two main components: the NYCFeatureEncoder and a multi-layer transformer encoder. The model is designed to recover masked features during pretraining, including numerical and categorical attributes such as latitude and longitude coordinates, trip time, medallion, and other metadata about the taxi trip.

The first component, the NYCFeatureEncoder, processes both numerical and categorical features. Numerical features, such as trip time, passenger count, and coordinates, are passed through a linear projection layer, while categorical features (e.g., medallion and hack license) are processed through separate embedding layers for each feature. During input, the masked values—determined by the masking strategy outlined in Section 3.1.4—are replaced by learnable tokens specific to each feature type. After encoding the features, the numerical and categorical features are concatenated and passed through a final linear output projection layer.

Next, the output from the feature encoder is passed to the TaxiMaskRecovery model, where it is positionally encoded using the formula proposed by Ashish Vaswani et al. [25]. The encoded data is then fed through three transformer encoder layers, utilizing multi-head attention to capture relationships and dependencies across different features. Finally, the output from these encoder layers is separated for the recovery of numerical features and individual categorical feature outputs.

During the pretraining process, the model is trained to minimize the prediction error of the masked features. The loss function combines the prediction errors for both categorical and numerical features, ensuring the model learns to recover each feature type accurately. A standard cross-entropy loss is used for categorical predictions, while a mean squared error loss is applied for numerical predictions. The model's accuracy is a combined result of its performance on both categorical and numerical features. For numerical features, a tolerance value is used to determine a valid recovery by calculating the difference between the actual and predicted values. Categorical feature accuracy is measured using the softmax of the probability vector.

In summary, the pretraining phase of the model is designed to learn rich representations of the NYC taxi data by predicting masked features. This enables the model to capture spatial, temporal, and trip-related dependencies, which can be leveraged in downstream tasks for more accurate predictions. The full code for all parts of the model, including the training code, is available on our GitHub repository.

3.2.2 - Downstream Tasks

This section focuses on why each task was chosen and what data was used for them, while the following section (3.2.3) discusses the details of their implementation.

To evaluate how well our model identified patterns in the data, we selected three tasks: predicting total trip duration, the pickup zone, and the drop-off zone. These tasks were well suited for evaluation because they corresponded to columns in the post-processed dataset that the model was trained on, and they tested different aspects of what we wanted the pretraining model to learn. Additionally, these tasks reflect practical applications of taxi trajectory modeling, such

as improving traffic forecasting, estimating travel times, and optimizing where and when taxis are dispatched.

3.2.2.1 – Trip Duration Model

The trip duration task aimed to predict the trip duration column using the remaining dataset features. To ensure the model learned the relationship between pickup-related features and trip duration, we excluded any drop-off-related columns to prevent data leakage. The selected input features included pickup coordinates, pickup time-based features (day, month, hour, minute, etc.), passenger count, trip distance, vendor ID, rate code, store-and-forward flag, and pickup zone. We applied numerical transformations, such as standard scaling for continuous variables, and encoded categorical features using a label encoder where applicable.

This task was useful because it assessed the model’s ability to relate various features to time-based columns. More importantly, trip duration is the feature most directly related to traffic conditions—holding all other factors constant, an increase in trip duration indicates higher congestion. The practical application of this model extends to real-world taxi services, where accurate trip duration predictions help improve estimated arrival times and route selection.

3.2.2.2 – Pickup Zone Model

The pickup zone prediction task evaluated the model’s ability to infer the pickup location without relying on pickup-related features. To prevent data leakage, we excluded columns such as pickup latitude, pickup longitude, pickup time features, and pickup zone. Instead, we used drop-off features—including drop-off time, drop-off coordinates—and general trip-related features such as trip distance, passenger count, vendor ID, store-and-forward flag, and drop-off zone.

This task was selected because pickup zone prediction is crucial for understanding spatial distribution in ride-hailing services. By accurately predicting pickup zones based on the available data, taxi companies and ride-sharing platforms can optimize driver allocation, anticipate peak demand areas, and improve overall efficiency. This has direct applications in demand forecasting and traffic management.

3.2.2.3 – Dropoff Zone Model

The drop-off zone model mirrored the structure of the pickup zone model but focused on predicting the drop-off location. To avoid data leakage, we excluded drop-off-specific columns, such as drop-off latitude, longitude, and time. The input features were similar to those used in the pickup zone model, except this time, we relied on pickup-related features (time, location, zone), trip distance, passenger count, and the same categorical attributes.

By training the model to predict drop-off zones independently of pickup-related features, we reinforced its ability to capture spatial patterns. This capability is essential for traffic optimization, where understanding how passengers move across different zones enables better route and traffic planning. Additionally, ride-sharing platforms can use drop-off zone predictions to facilitate ride-pooling, anticipating requests with similar destinations more effectively.

3.2.3 - Baseline Models

To properly evaluate how effective our pretraining model was at completing downstream tasks we created neural networks only using the small training and testing dataset dedicated for these tasks. This serves as a baseline to compare the finetuning tasks of the pretraining model to. Ultimately, we aimed to show that with a relatively small amount of data, the pretrained model would be able to significantly outperform these baseline models. The architecture of these three

models were each three layer neural networks where the trip duration model's output was a single value representing the predicted duration, and the pickup and drop-off zone models' outputs were a vector of values that are passed through SoftMax to get the predicted probability for each zone. We decided to use a simple 2 hidden layer neural network each with 256 nodes to attempt to model each task.

Our next step was to experiment with multiple hyperparameters to optimize the model and address potential overfitting due to the limited data. First, learning rate is how much the weights change after each iteration. A learning rate too small may not find the best weights quickly, and a learning rate too large may overshoot the best weights and not be able to find them. Next, batch size is the number of samples before updating weights. A batch size too small may not accurately update the weights and increase training time, and a batch size too large updates weights less often and may not find the best weights. Then, weight decay is regularization term that penalizes weights that are too large. A weight decay too small will allow overreliance on a single column, and a weight decay too large will create weights that only care about being small. Finally, dropout layers are layers after each hidden layer that randomly set the output of nodes to zero. This allows the model to generalize better with the risk of losing information due to them. We used 80% of our training dataset to create a subset and performed 5-fold cross validation. Then, we used grid search, which tried each combination of hyperparameters shown in Figure 1 and evaluated them, on that subset to find the best ones.

Hyperparameters Tested	Pickup Zone Model			
Learning Rate	0.05	0.01	0.005	
Batch Size	16	32	64	
Weight Decay	0.0	0.05	0.1	0.5

Dropout	0.0	0.2
---------	-----	-----

Figure 1: Hyperparameter Testing

The last overfitting remedy we included was early stopping. This occurred after the hyperparameter tuning and was measured using the validation dataset taken before hyperparameter tuning that contained 20% of the training dataset. After each training epoch, in addition to the training metric (either loss or accuracy), the metric was also calculated based on the validation dataset. This value was not used in back propagation or in any other way to directly affect the training of the model, but was used to find the best number of epochs to use in case too many caused overfitting. Each validation metric was compared to the one from the previous epoch and if three consecutive epochs resulted in no improvement in the validation metric, early stopping triggered and stopped training at that epoch. That model then was then used on the test data with the hope it would not overfit.

4 – Results

Section 4 details the results of the implemented models as described in Section 3.2, utilizing the data described in Section 3.1. Our goal, as described in the previous section, is to pretrain a large model using mask and recovery for later use in various fine-tuning tasks. The following results and analysis will cover three areas, Pretraining Results (see Section 4.1), Baseline Model Results (see Section 4.2), and Fine-tuning Results (see section 4.3).

4.1 – Pretraining Results

Throughout the pre-training stage, a multitude of experiments were conducted to determine which features to mask and when. These experiments explored various combinations

of masked features—such as pickup and drop off coordinates, time-related variables, and categorical identifiers—to evaluate their impact on the model's ability to accurately recover missing information. The results were carefully analyzed to determine what pairs of features performed best together, while also ensuring that the pre-training would be useful for our fine-tuning tasks. While we will primarily focus on the results that led to successful pre-training in this section, a host of other unsuccessful experiments can be found on our GitHub repository.

4.1.1 - Masking All Numerical Features

One of the initial sets of experiments conducted after finalizing the model architecture involved randomly masking and recovering all numerical features. As outlined in Section 3.1 – Data, the dataset comprised 17 distinct numerical features, each presenting unique challenges and varying levels of recovery accuracy.

Through extensive experimentation, the optimal configuration was determined to be a model dimension of 256, a learning rate of 0.0001, a batch size of 64, a tolerance of 0.1, and a dataset size of approximately 14 million trips. Interestingly, model dimension (within the range of 128–256) and batch size (64–256) had minimal impact on performance. In contrast, learning rate and tolerance were found to be critical factors. Lower learning rates resulted in prohibitively slow training, whereas higher rates led to overfitting and substantial drops in accuracy.

Tolerance played a crucial role in evaluating the success of masked feature recovery. Defined as the acceptable difference between the predicted and actual values, it effectively set the threshold for a valid recovery. Increasing the tolerance consistently improved accuracy by allowing a wider margin for acceptable predictions, while lowering it made the task more difficult, reducing accuracy and making training more difficult.

It is important to note that the results presented in the following figure were not achieved on the first attempt. Instead, they are the product of multiple iterations of fine-tuning hyperparameters, as described above.

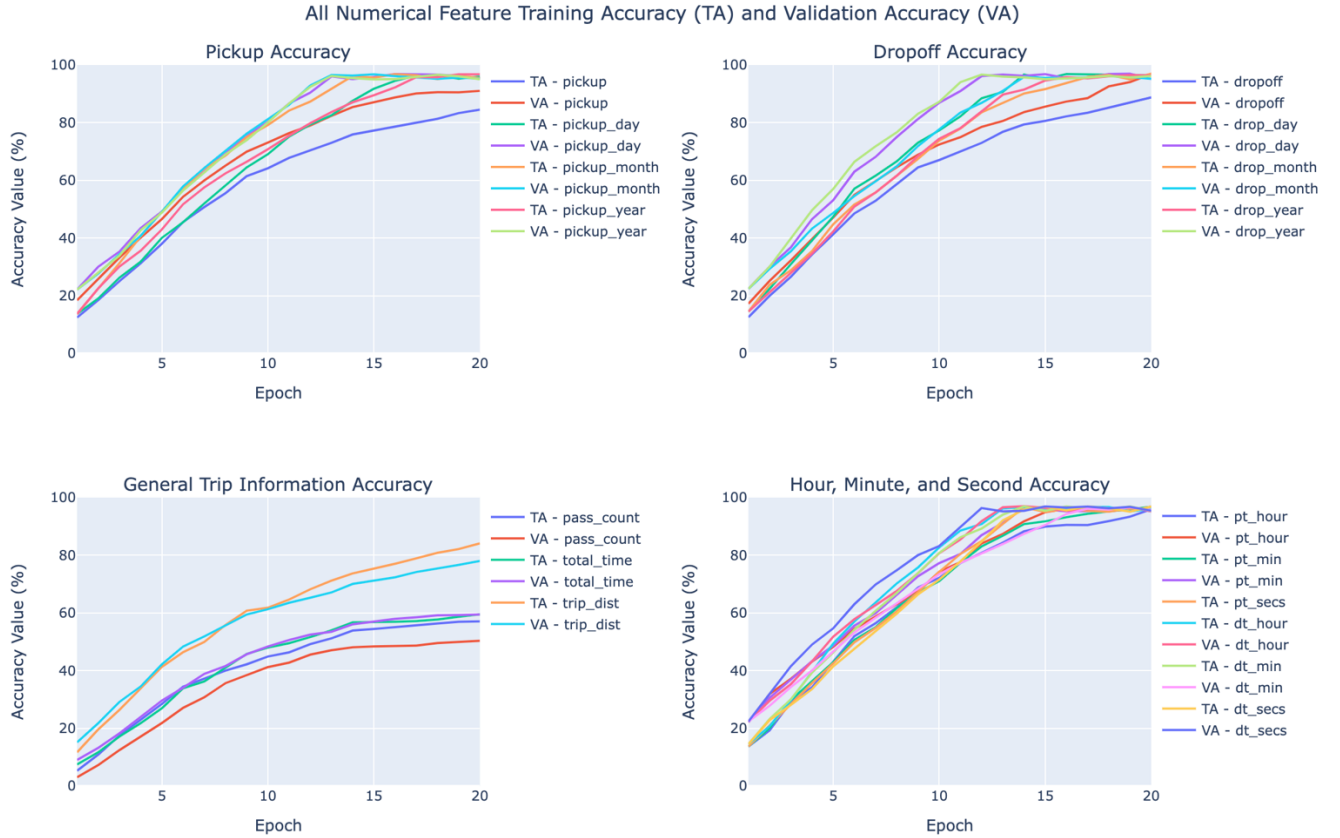


Figure 2: Numerical pretraining Feature Accuracies

As shown in Figure 2, this experiment achieved strong results across most features, with a final aggregate accuracy of 90.87%. Time-related features, such as day, month, and year, performed exceptionally well, each reaching approximately 95%–97% accuracy—likely due to their limited data ranges. The model also recovered pickup and drop off coordinates effectively, achieving around 85% accuracy for each. However, some features underperformed, most notably passenger count (56%) and total time (60%). While both initially showed steady improvement,

their accuracy plateaued around epoch 15, likely due to overfitting on simpler features like pickup day and drop off day, which the model learned more easily.

4.1.2 - Masking Numerical and Categorical Features

The next set of experiments involved masking almost all features in our dataset; All 17 numerical features and four categorical features. Three categorical features, rate code, store and forward flag, and vendor id were excluded due to their very limited number of unique values, which led to the model overfitting to those features and hampering the learning of the others. Most importantly, considering that two of our downstream tasks were related to zones, we needed to experiment masking the pickup and drop off zones alongside the numerical features.

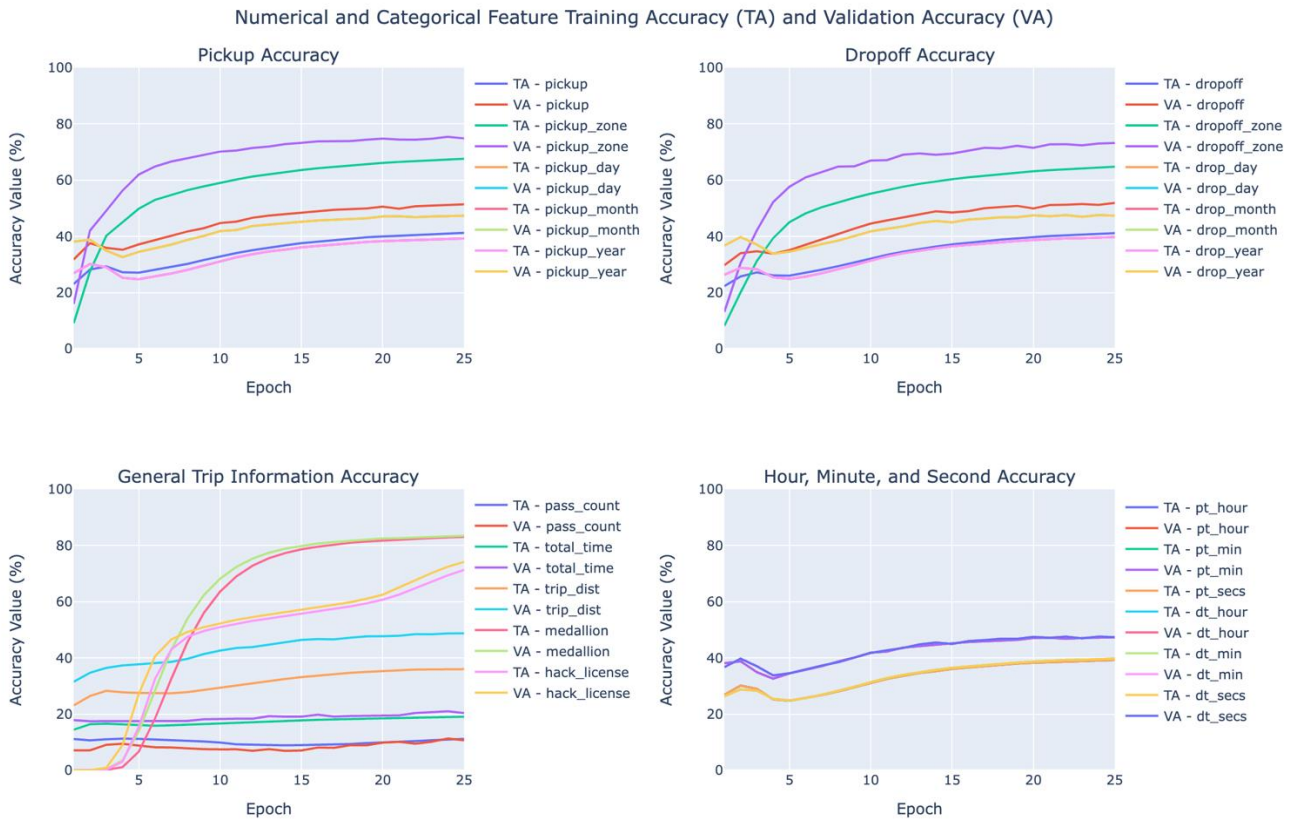


Figure 3: Numerical and Categorical pretraining Feature Accuracies

Like the model in Figure 2, the model for Figure 3 used similar hyperparameters: a model dimension of 256, a learning rate of 0.0001, a batch size of 128 (which helped speed up training without significantly impacting accuracy), a tolerance of 0.1 for numerical features, and a dataset size of approximately 14 million trips. However, the results deviated drastically from our initial experiments focused solely on numerical features.

Although the model's structure remained relatively unchanged, performance on the numerical features in Figure 3 was notably poorer. The best-performing numerical feature, pickup time in seconds, achieved only around 50% accuracy, a significant drop from the 97% accuracy it had in the previous model. Furthermore, while the broken-down time features showed some gradual improvement during training, more complex numerical features, such as passenger count and total trip time, stagnated, with accuracies lingering around 20% and 10%, respectively. In contrast, accuracy on the categorical features improved steadily throughout training. Medallion accuracy peaked at over 80%, and pickup and drop-off zone validation accuracy reached approximately 75%. This steady growth in categorical feature performance suggests a positive trajectory for fine-tuning the model in later stages for zone prediction.

Despite this, the model's focus on categorical features may have contributed to overfitting, where the model became overly specialized in learning patterns in the categorical data. This could have hindered the learning of the more nuanced numerical features, as the model may have allocated too much capacity to capturing categorical information, leaving insufficient resources for the numerical features to improve adequately. This overfitting to the categorical features likely explains the stagnation in accuracy for the more challenging numerical features and highlights the need for a more balanced model or techniques to regulate categorical feature learning.

4.1.3 - Masking Subsets of Features

The final set of experiments involved masking different subsets of features to boost the overall mask and recovery accuracy of the model, while ensuring enough features were included to support the later fine-tuning stage. The following model results used the same hyperparameters as the model in Figure 3.

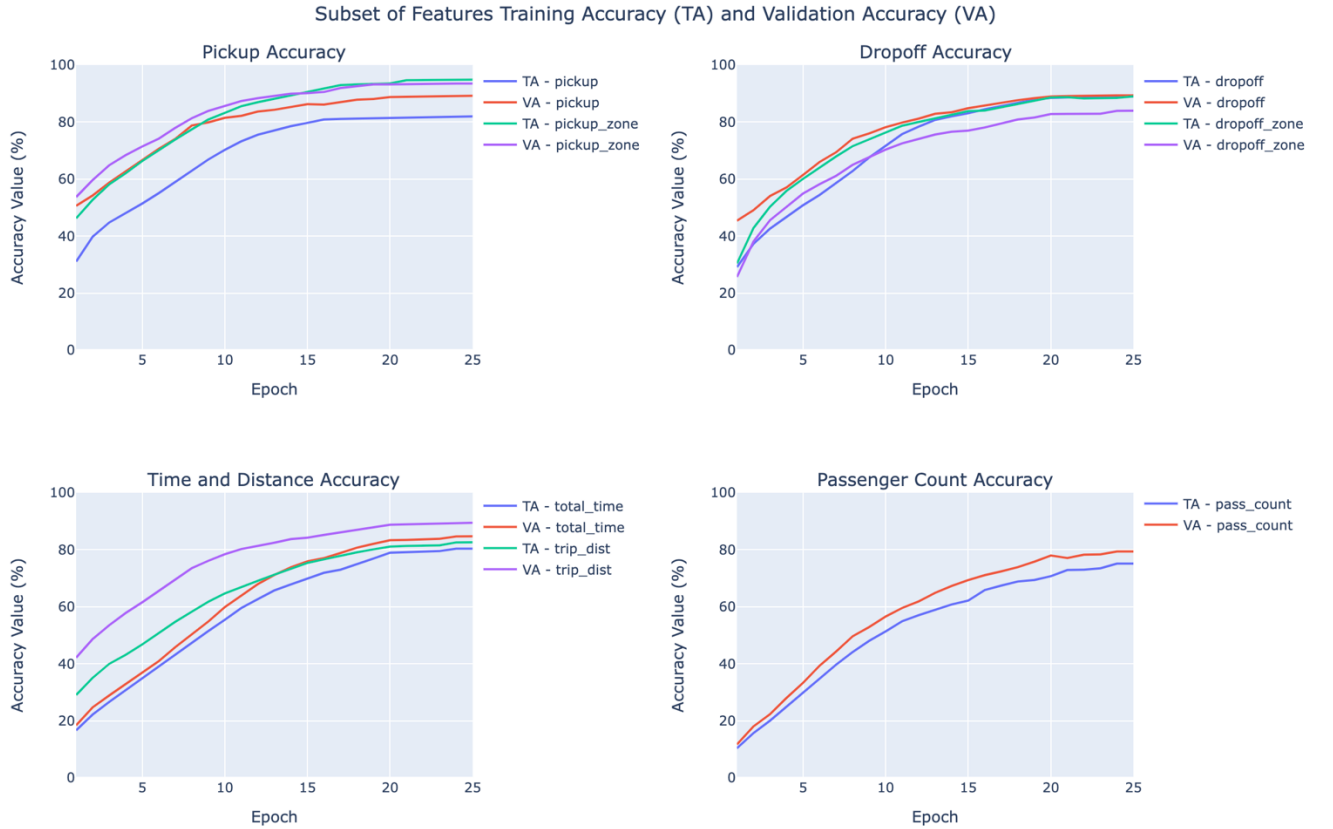


Figure 4: Subset of Features pretraining Accuracies

The results shown in Figure 4 align closely with our desired goals, achieving a high aggregate accuracy of 87.68%. More importantly, the model performed exceptionally well on the key features required for fine-tuning. Pickup and drop-off zone recovery were the strongest, with pickup zone validation accuracy reaching 93.50%, outpacing drop-off zone accuracy at 84%.

Additionally, total time recovery saw a significant improvement from Figure 3, reaching 80.38% accuracy on training data and 84.7% on validation data. This boost in accuracy can likely be attributed to several factors. First, easier-to-learn numerical features, such as pickup day, month, and year, were excluded from masking, allowing the model to focus more on the challenging numerical and categorical features. Second, particularly difficult categorical features like medallion and hack license were also excluded, refocusing the model on the features essential for fine-tuning. While these features may provide valuable information, their inclusion had previously hindered the recovery of numerical features, making their removal a necessary trade-off.

With this strong pretraining performance, the model is well-prepared for fine-tuning on downstream tasks. The learned feature representations serve as a solid foundation for producing the results presented in one of the following sections, 4.3 – Fine tuning Results, where we evaluate the model’s effectiveness on improving the loss and accuracy on three different downstream tasks.

4.2 – Baseline Model Results

These are the neural networks we created that attempted to model our three downstream tasks of trip duration, pickup zone, and drop-off zone. As detailed in section 3.2.3, these models were intended to be a benchmark for how well a model without any pretraining could perform on various problems with limited data. Experiments were performed to find the best hyperparameters in order to optimize the model to the best of our ability. These results were then analyzed to understand how well they achieved their intended purpose and could then be used as a comparison point to our fine-tuning models.

4.2.1 - Trip Duration Model

After performing hyperparameter tuning as described in section 3.2.3, the best hyperparameters were a learning rate of 0.005, batch size of 16, and a weight decay of 0. This resulted in a train loss of 35458.1686 and a test loss of 115325.691, with more results available in Appendix I. Additionally, when a dropout rate of 0.2 was used, the best hyperparameters were a learning rate of 0, batch size of 32, and weight decay of 0. Using dropout had worse training and marginally better testing loss than without dropout. The train loss was 39507.3352 and the test loss was 104070.457 with neither of these models triggered early stopping. These large amounts of test loss show that the task of learning trip duration is not feasible with an independent model using this small amount of data. Also, because the best weight decay was 0, dropout marginally improved the model, and there was no early stopping, it indicates that overfitting the training data was not necessarily the issue.

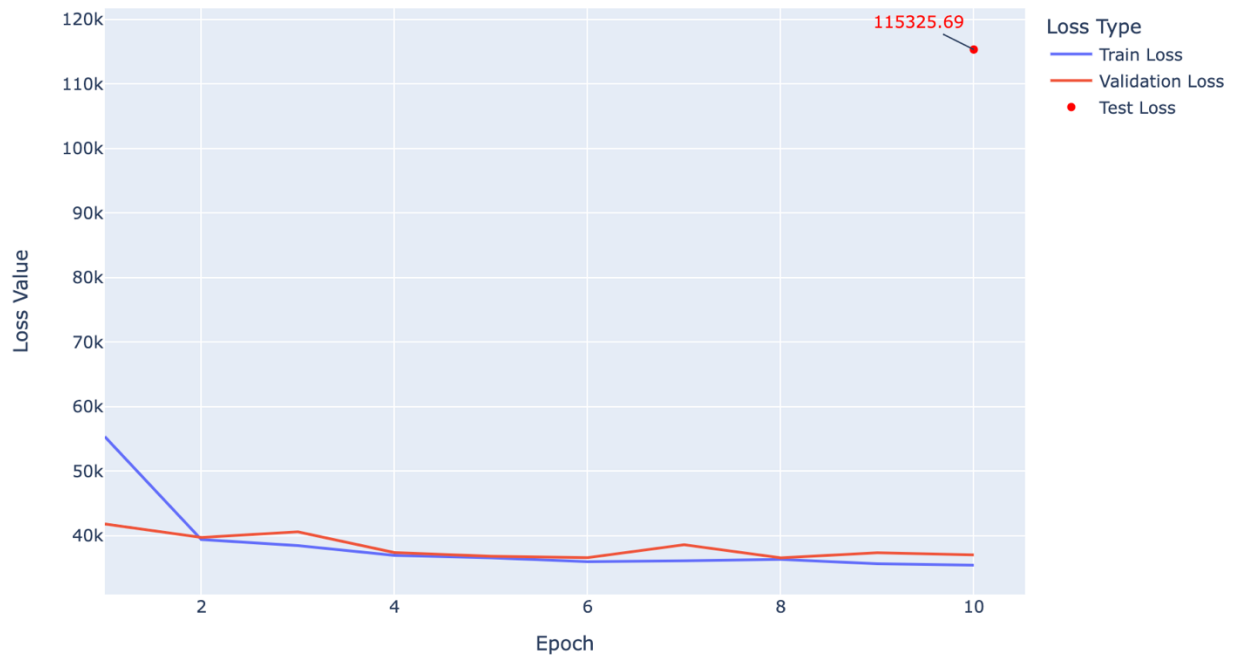


Figure 5: Train and validation loss for each of ten epochs and final test loss for the baseline trip duration model

4.2.2 – Pickup Zone Model

The test dataset had 248 unique pickup zones concentrated around the ANK1089 zone with total pickup counts decreasing further away from it, as can be seen in figure 6. Some zones had over a thousand datapoints and some only had a handful. Hyperparameter tuning for the pickup zone model resulted in a learning rate of 0.001, batch size of 32, and a weight decay of 0. These were the same when implementing a dropout of 0.2 as well. The best model had a train accuracy of 23.03%, and a test accuracy of 21.32%, which can be referenced from Appendix II. Both of these models ran for the full 10 epochs without early stopping. This model did not perform significantly worse on the test set compared to the train set, but both the training and testing did not accurately predict a large amount of pickup zones. Although it may be pointed out that there are 248 different pickup zones meaning our model did better than a random guess

model, this may not be a good baseline. From figure 7, the model only had mediocre accuracy in the most common zones and high accuracy only when it predicted correctly in a zone with few classes. Additionally, as seen in figure 8 the train, validation, and test accuracies remained relatively steady throughout the epochs showing that they were not gaining significant knowledge during them. This indicates that the model can only learn patterns in a select few zones, which is not useful in the real world application because it does not understand zones on the outskirts of New York. This reinforces the idea that this baseline model is unable to properly achieve its intended goal due to the limited data.

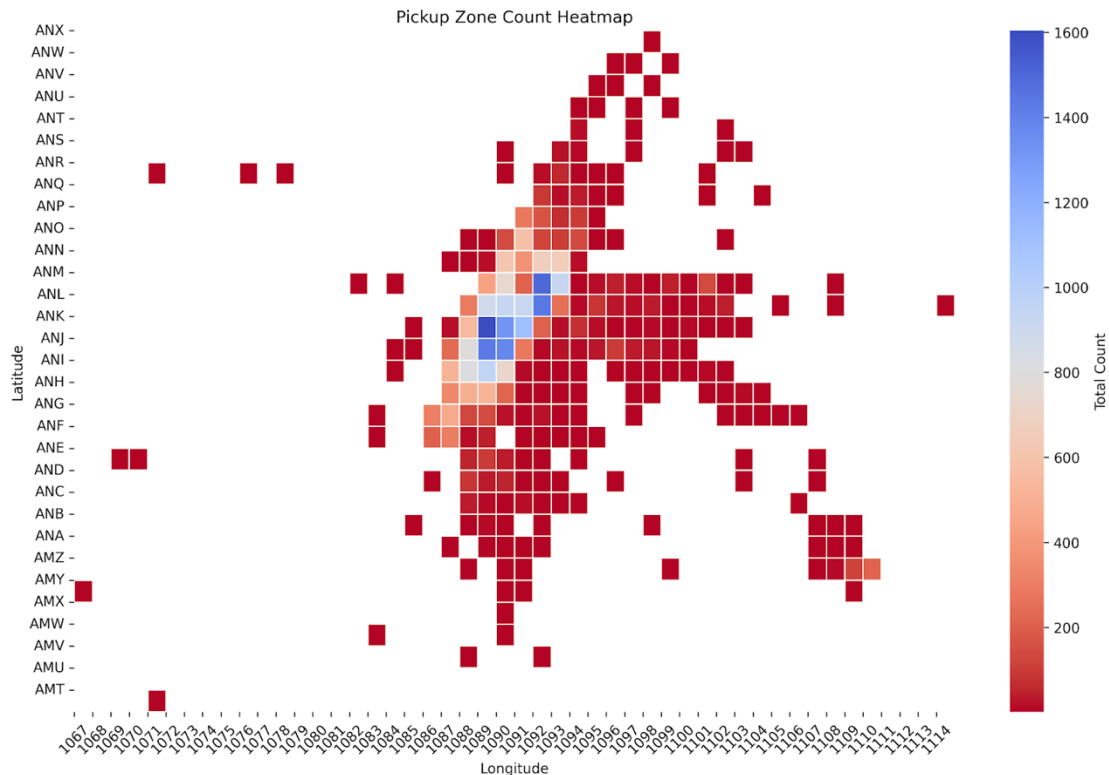


Figure 6: Heatmap of the total number of data points in each pickup zone in the test dataset

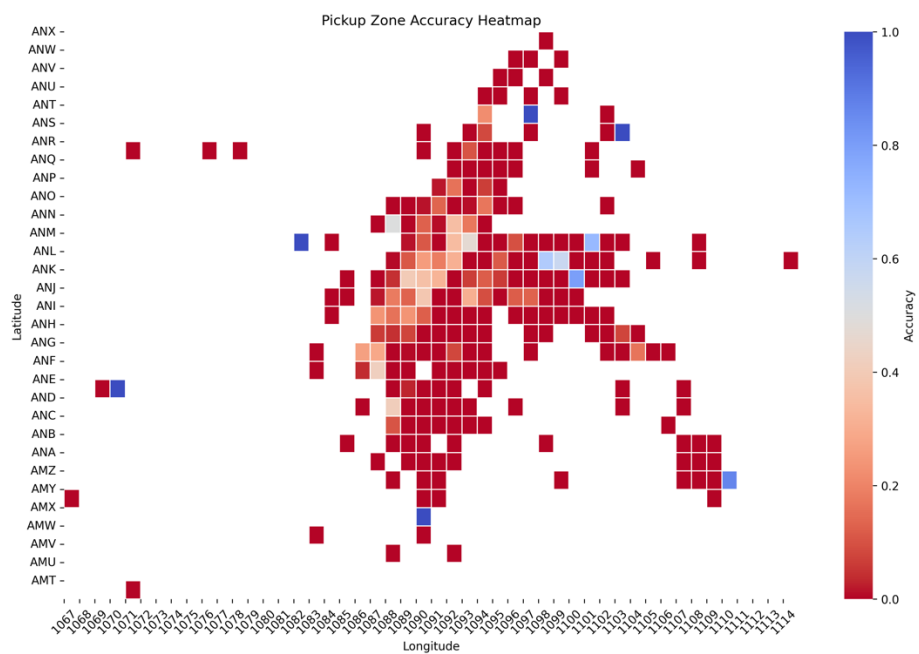


Figure 7: Heatmap of the baseline model's accuracy in predicting each pickup zone in the test dataset

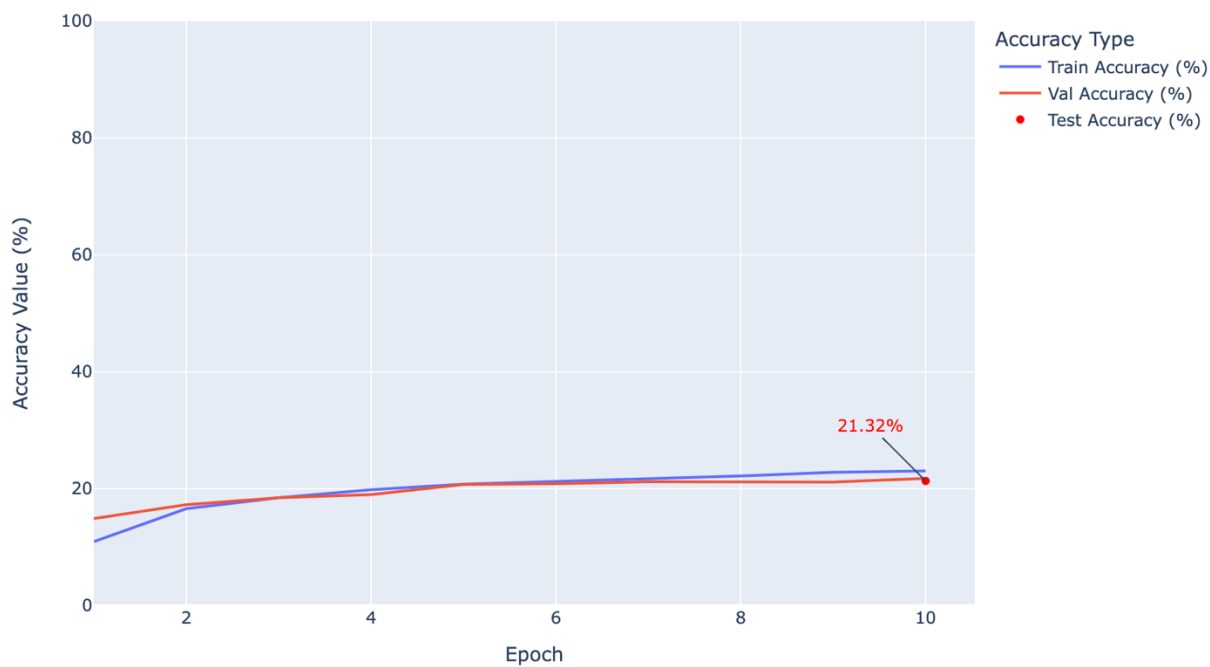


Figure 8: Train and validation accuracy for each of ten epochs and final test accuracy for the baseline pickup zone model

4.2.3 – Dropoff Zone Model

The drop-off zone model had similar results to the pickup zone model. The hyperparameter tuning for the model without dropout resulted in a learning rate of 0.001, batch size of 32, and a weight decay of 0. The best parameters for the model with dropout was a learning rate of 0.001, batch size of 64, and a weight decay of 0. Neither of these models triggered early stopping. Between these two, the model performed better without dropout with a train accuracy of 25.07% and a test accuracy of 27.12%, compared to a test accuracy of 26.50% from the one with dropout. Complete results can be seen in Appendix III. The test dataset contained 425 unique drop-off zones and from figure 9, they are primarily centered around zone ANK1089 in the same fashion found in drop-off counts. For the same reason as in section 4.2.2, this model also should not be evaluated just compared to a random guess model because of the uneven class distribution. Also as seen in figure 10, the model only achieved high accuracy in zones with few data points as it did in the pickup zone model. The accuracies shown in figure 11 also indicate that the model did not substantially learn throughout the training epochs. This led us to the same conclusion, that this model, and subsequently all of the baseline models, could not capture the patterns in the data that would be required to draw practical applications from.

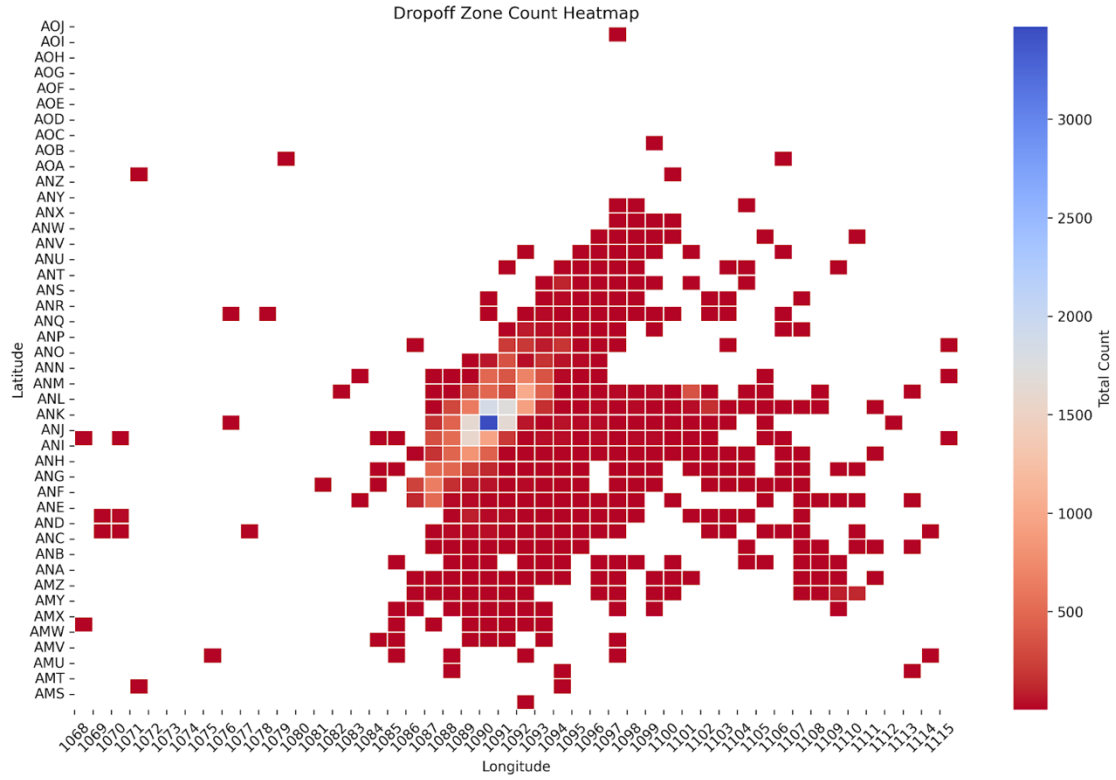


Figure 9: Heatmap of the total number of data points in each dropoff zone in the test dataset

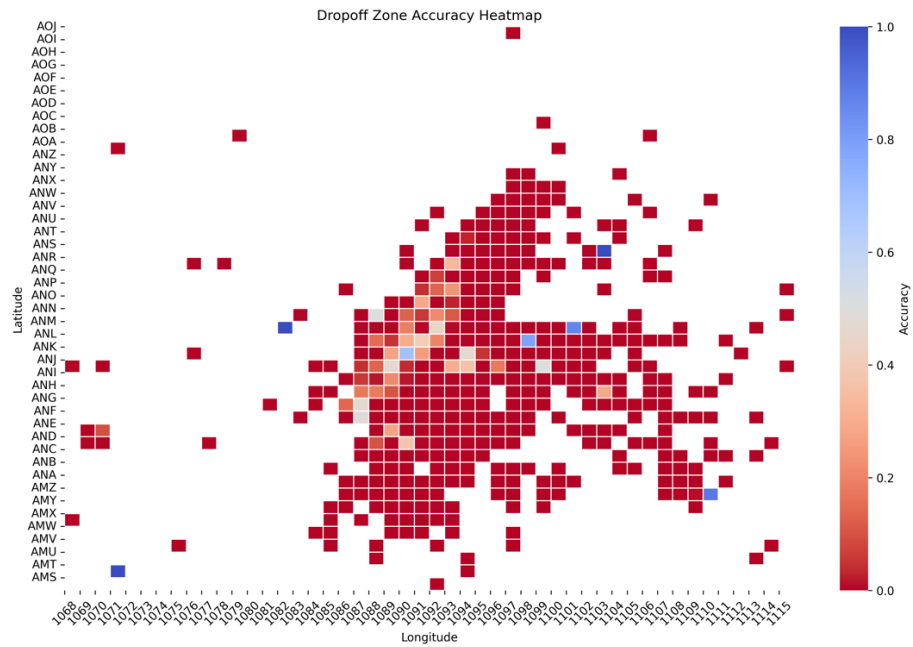


Figure 10: Heatmap of the baseline model's accuracy in predicting each pickup zone in the test dataset

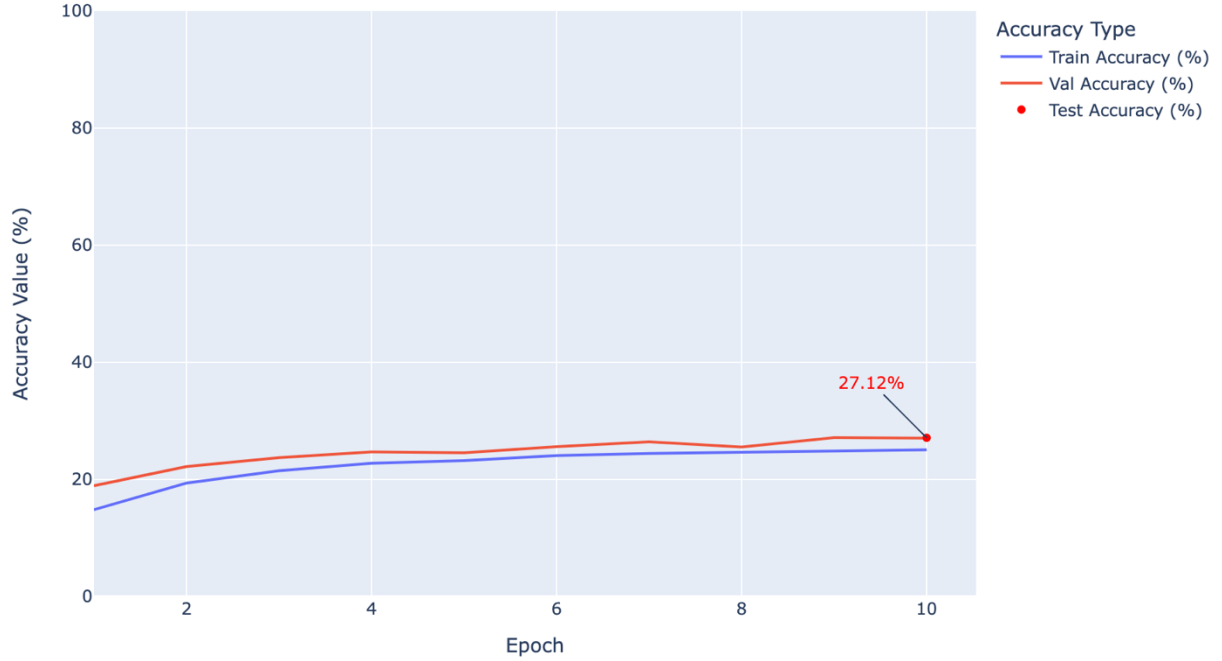


Figure 11: Train and validation accuracy for each of ten epochs and final test accuracy for the baseline dropoff zone model

4.3 - Fine tuning Results

In the previous section, we evaluated the performance of standalone baseline models for trip duration, pickup zone, and drop-off zone prediction. While these models were trained well considering the small amount of data provided to them, they wouldn't necessarily be useful in a real-world scenario because of this. In this section, we extend our analysis by incorporating our transformer encoder model. We compare two variations: a model with randomly initialized weights and a pretrained model with learned weights. These transformer encoders serve as the initial input layers, with the previously tested baseline models attached to their output. By comparing results across these setups, we assess the impact of pretraining on downstream task performance. Across all the following results, the weights of the transformer encoder, whether pretrained or not, were allowed to be modified during the fine-tuning training process.

4.3.1 - Trip Duration Model

As described in Section 4.2.1, our baseline trip duration model ended with loss values between 35,000 and 40,000, with the final test loss being approximately three times higher. However, incorporating our transformer encoder model significantly improved performance, reducing the loss to a fraction of its original value.



Figure 12: Fine-tuned Duration Model Results

As shown above in Figure 12, the loss drastically decreased from the tens of thousands seen in Figure 5 to well below one. The transformer encoder model yielded strong improvements, reaching a final test loss of 0.1303 with random weights and 0.0886 with pretrained weights. The full loss results at each epoch can be found in Appendix IV. While both versions of the transformer model significantly improved performance, the pretrained model consistently outperformed the randomly initialized version across all three datasets. On the training set, pretraining reduced loss by a factor of 11.7, and on the validation set, it achieved an

even greater reduction of 22.4 in the final epoch. This suggests that pretraining helped the model develop a stronger understanding of the underlying data structure, leading to better generalization and more accurate trip duration regression.

4.3.2 – Pickup Zone Model

The standalone pickup zone baseline model ended with a test accuracy of 21%, with training and validation accuracies at 23% and 21%, respectively. Using the same data—where pickup zone counts are distributed as shown in Figure 6—we incorporated our transformer encoder model to significantly enhance zone prediction accuracy.

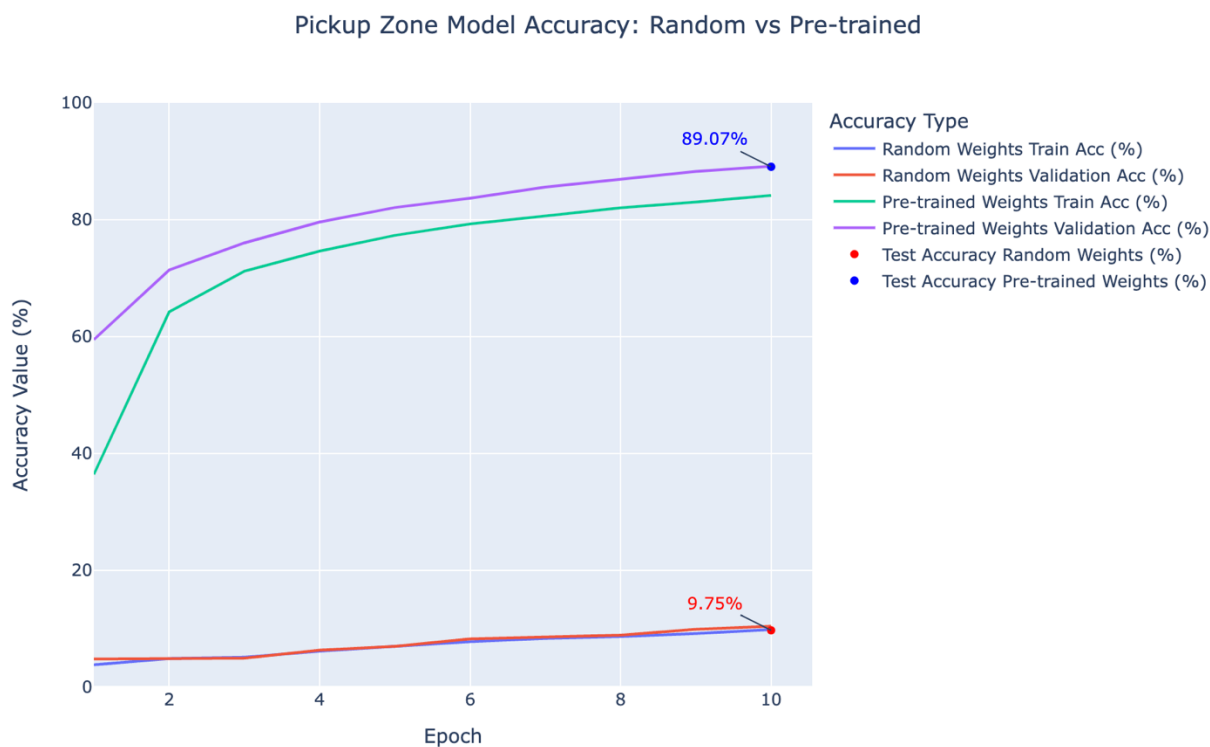


Figure 13: Fine-tuned Pickup Zone Model Results

Displayed above in Figure 13 are the differences in pickup zone prediction accuracy between the randomly initialized and pretrained models. Similar to the results in Section 4.3.1,

the pretrained model consistently outperformed the random model, achieving a final test accuracy of 89.07%—a nine-fold increase compared to the random model's 9.75%.

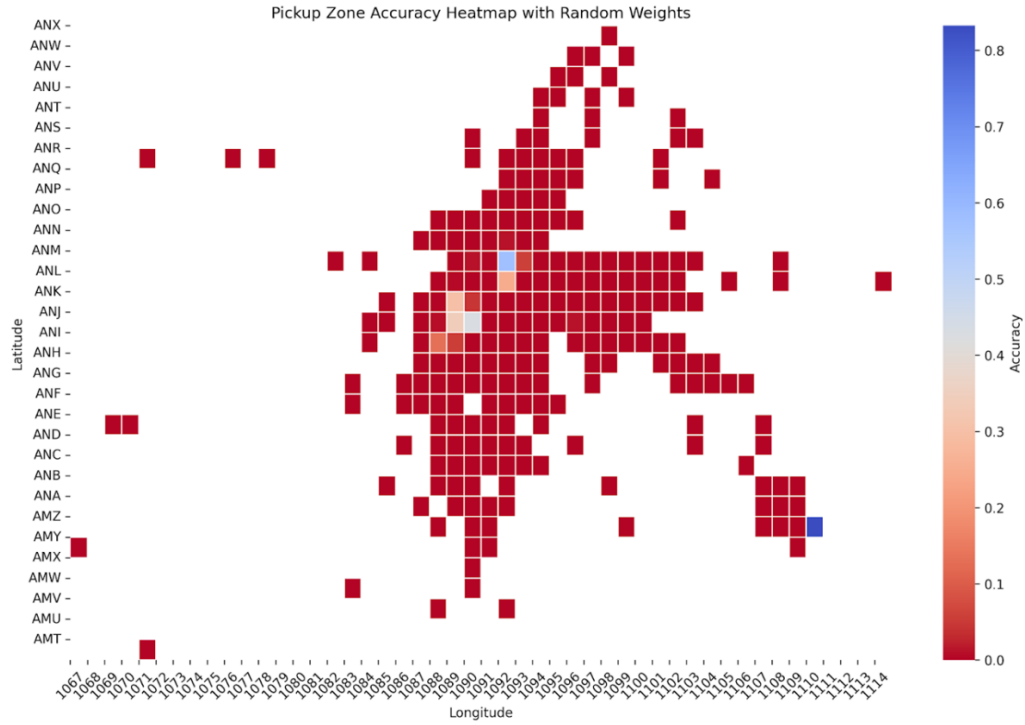


Figure 14: Fine-tuned Pickup Zone Accuracy Heatmap using randomly initialized weights

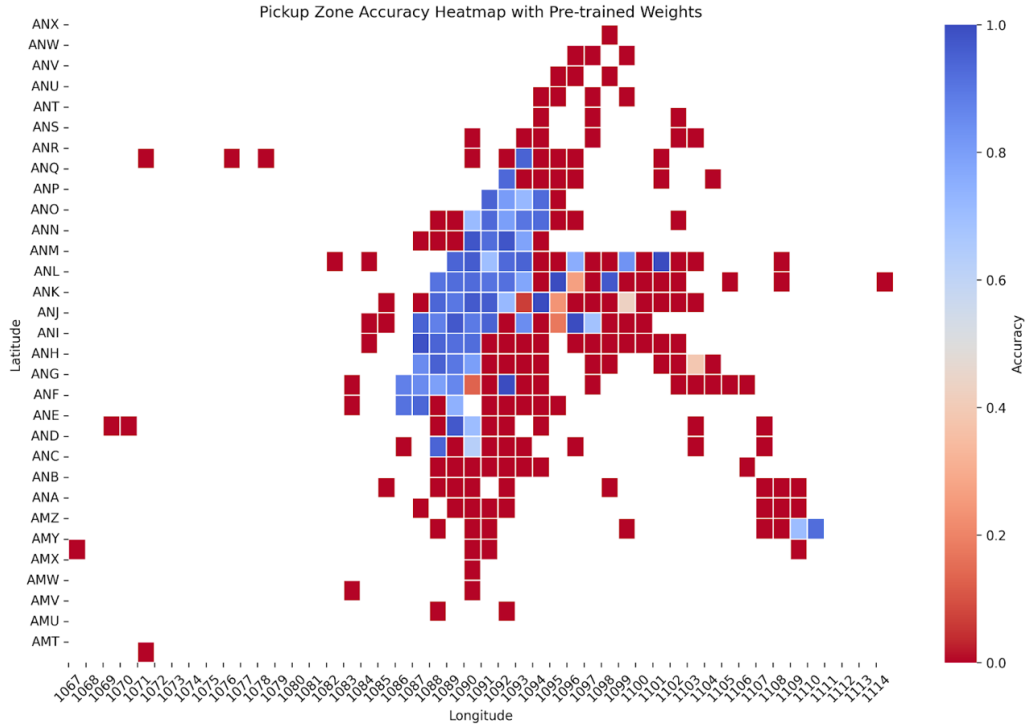


Figure 15: Fine-tuned Pickup Zone Accuracy Heatmap using Pre-trained Weights

This stark contrast is further illustrated in Figures 14 and 15, which highlight the improved accuracy in regions where our data was most concentrated. The full accuracy values at each epoch can be found in Appendix V.

However, in this case, the randomly initialized model performed worse than the standalone baseline model, suggesting that the transformer encoder was too complex to train effectively with limited data, leading to reduced accuracy. Despite this, the pretrained model significantly outperformed both, reaching 89% accuracy compared to the standalone model's 21%. These results further emphasize the effectiveness of pretraining in helping the model develop a better general understanding of pickup zone distributions, resulting in superior prediction accuracy.

4.3.3 – Dropoff Zone Model

The drop-off zone baseline model achieved a test accuracy of 27.12%, with training and validation accuracies at 25% and 27%, respectively. Using the same dataset—where drop-off zone counts follow the distribution shown in Figure 9—we integrated our transformer encoder model to substantially improve zone prediction accuracy.

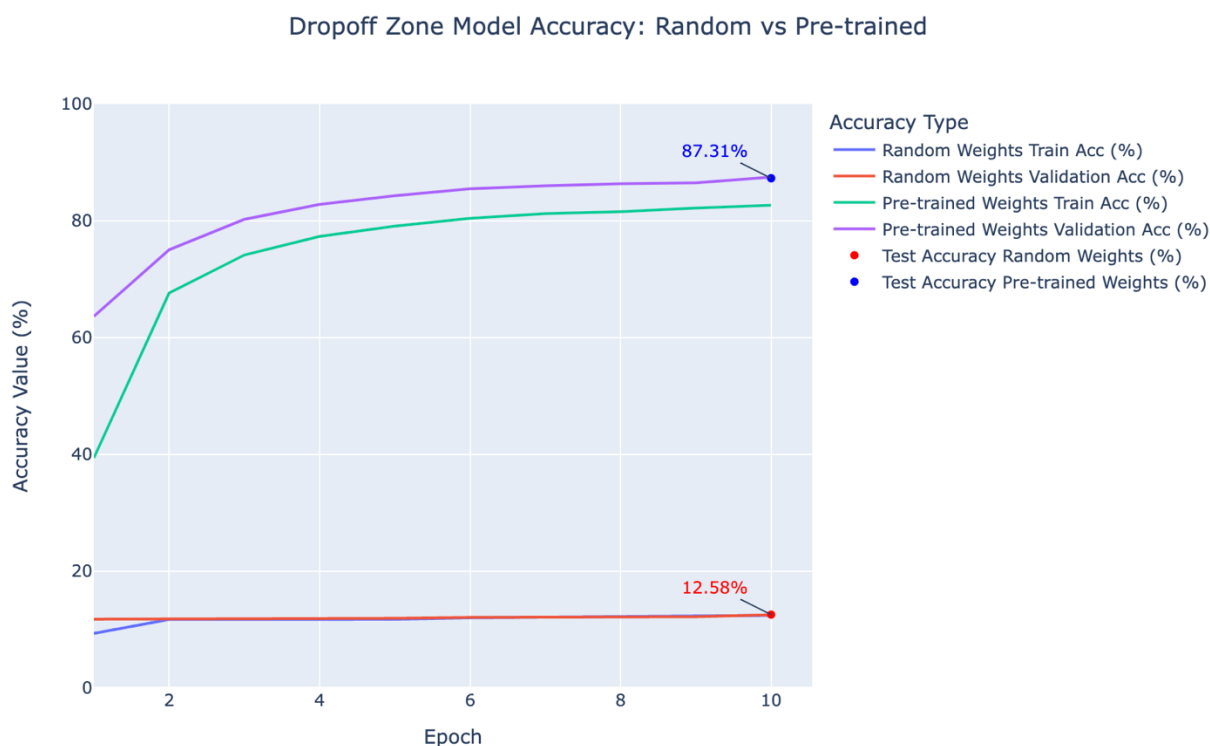


Figure 16: Fine-tuned Dropoff Zone Model Results

Figure 16 illustrates the differences in drop-off zone prediction accuracy between the randomly initialized and pretrained models. As seen in the previous subsections (4.3.1 and 4.3.2), the pretrained model consistently outperformed the random model, reaching a final test accuracy of 87.31%—a nearly seven-fold improvement over the random model's 12.58%.

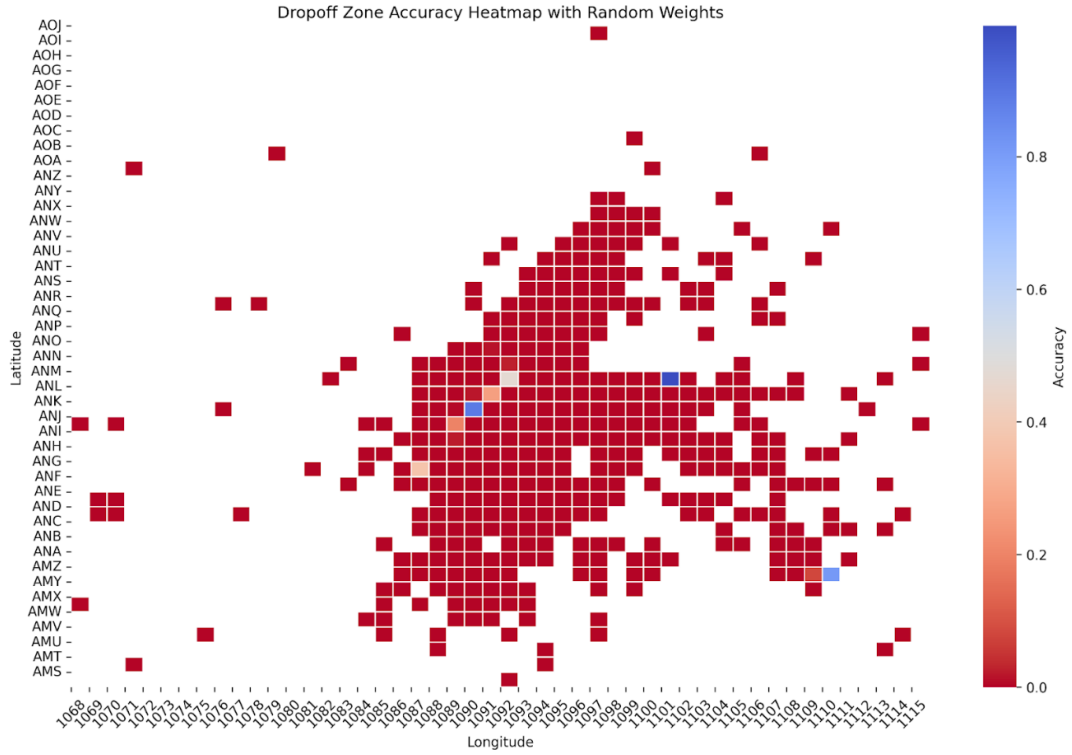


Figure 17: Fine-tuned Dropoff Zone Accuracy Heatmap using randomly initialized weights

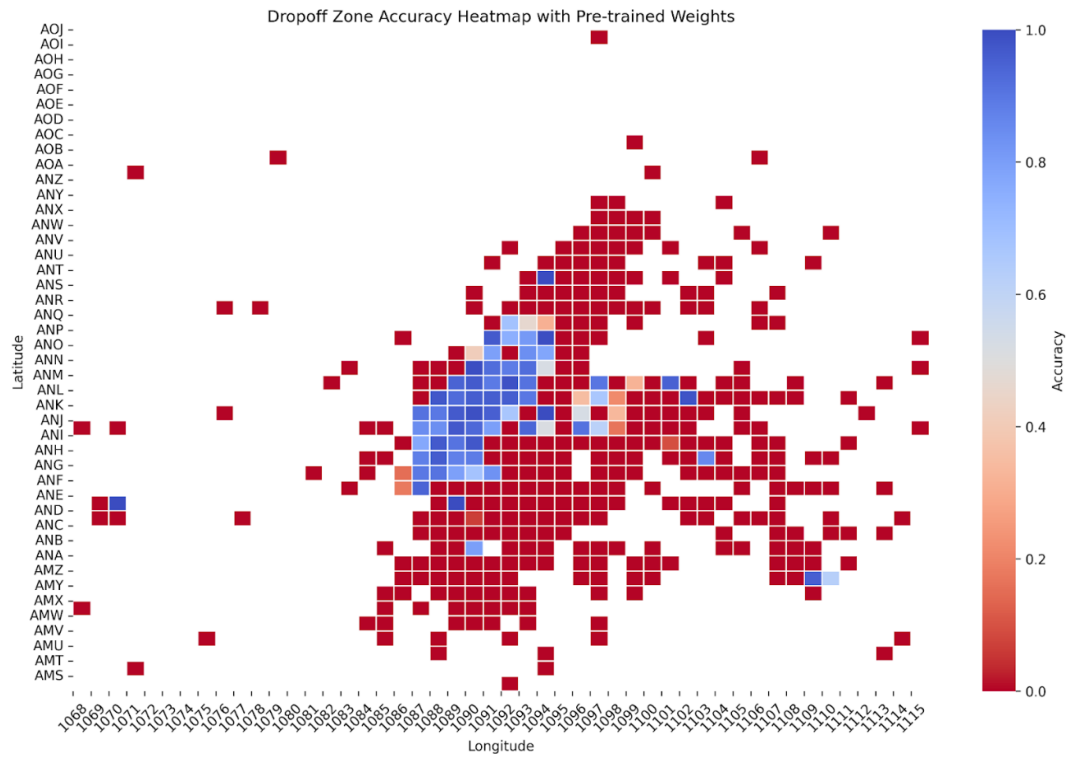


Figure 18: Fine-tuned Pickup Zone Accuracy Heatmap using Pre-trained Weights

The substantial difference is further demonstrated in Figures 17 and 18, which showcase the enhanced accuracy in regions with the highest data concentration. While the pickup and drop-off zone distributions differed, the results align with those of pickup zone prediction, as discussed in Section 4.3.2. Full accuracy values for each epoch are provided in Appendix VI.

Similar to the fine-tuned pickup zone model, the randomly initialized model performed worse than the standalone baseline model, reinforcing the idea that the transformer encoder was too complex to train effectively with limited data. Nevertheless, the pretrained model significantly outperformed both, achieving an accuracy of 89% compared to the standalone model's 27.12%. These findings further highlight the impact of pretraining in enabling the model to develop a stronger general understanding of drop-off zone distributions.

5 – Conclusions

In conclusion, throughout our research we were able to prove the viability of a foundation model based on spatial-temporal data. By developing a baseline model as well as a fine-tuning model for each of the downstream tasks we created, we displayed the benefit of using a foundation model in the form of increased performance and decreased loss for the fine-tuning outputs. The three downstream tasks were able to display collectively that this type of model can be applied in many situations and should be further developed. In future research, other downstream tasks and data input types should be explored. By being able to cover a wide variety of tasks and receive various types of data, the foundation model would be applicable to a wider range of models.

6 - Appendix

Appendix I

Trip Duration Baseline Model Results

	Without Dropout Layers			With 0.2 Dropout Layers		
Best Hyperparameters	LR=0.005, Batch Size=16, Weight Decay=0			LR=0.001, Batch Size=32, Weight Decay=0		
Epoch	Train Loss	Validation Loss	Test Loss	Train Loss	Validation Loss	Test Loss
1	55369.229	41822.8022	n/a	56875.4512	41195.892	n/a
2	39422.8057	39761.9857	n/a	44937.6382	38834.3809	n/a
3	38481.632	40622.5737	n/a	42530.9569	47425.4993	n/a
4	36982.5656	37397.8808	n/a	41578.6806	38095.2543	n/a
5	36590.1	36836.5323	n/a	41297.2171	37680.8553	n/a
6	36007.2849	36608.7914	n/a	40881.1979	39588.5792	n/a
7	36121.0805	38628.6559	n/a	40624.1678	38699.2853	n/a
8	36361.7001	36588.4007	n/a	40188.9277	37190.599	n/a
9	35675.0671	37378.3246	n/a	40236.3502	36653.8017	n/a
10	35458.1686	37058.5345	115325.691	39507.3352	36889.6577	104070.457

Appendix II

Pickup Zone Baseline Model Results

	Without Dropout Layers	With 0.2 Dropout Layers
Best	LR=0.001, Batch Size=32, Weight Decay=0	LR=0.001, Batch Size=32, Weight Decay=0

Hyperparameters						
Epoch	Train Acc	Validation Acc	Test Acc	Train Acc	Validation Acc	Test Acc
1	0.1093	0.1488	n/a	0.0992	0.1351	n/a
2	0.1657	0.1725	n/a	0.1446	0.1605	n/a
3	0.1844	0.1844	n/a	0.1646	0.1847	n/a
4	0.1981	0.1899	n/a	0.1748	0.1966	n/a
5	0.2078	0.2070	n/a	0.1828	0.2021	n/a
6	0.2122	0.2082	n/a	0.1891	0.2026	n/a
7	0.2175	0.2118	n/a	0.1918	0.2110	n/a
8	0.2217	0.2107	n/a	0.1977	0.2117	n/a
9	0.2279	0.2112	n/a	0.1966	0.2062	n/a
10	0.2303	0.2176	0.2132	0.2007	0.2095	0.2091

Appendix III

Dropoff Zone Baseline Model Results

	Without Dropout Layers			With 0.2 Dropout Layers		
Best Hyperparameters	LR=0.001, Batch Size=32, Weight Decay=0			LR=0.001, Batch Size=64, Weight Decay=0		
Epoch	Train Acc	Validation Acc	Test Acc	Train Acc	Validation Acc	Test Acc
1	0.1484	0.2893	n/a	0.1444	0.1921	n/a
2	0.1938	0.2221	n/a	0.1868	0.2211	n/a
3	0.2150	0.2373	n/a	0.2073	0.2382	n/a
4	0.2277	0.2469	n/a	0.2204	0.2427	n/a

5	0.2321	0.2455	n/a	0.2291	0.2485	n/a
6	0.2408	0.2560	n/a	0.2351	0.2547	n/a
7	0.2443	0.2644	n/a	0.2385	0.2563	n/a
8	0.2462	0.2554	n/a	0.2450	0.2544	n/a
9	0.2497	0.2713	n/a	0.2466	0.2651	n/a
10	0.2507	0.2704	0.2712	0.2504	0.2686	0.2650

Appendix IV

Trip Duration Fine Tuning Model Results

	Random Weights			Pre-trained Weights		
Epoch	Train Loss	Validation Loss	Test Loss	Train Loss	Validation Loss	Test Loss
1	0.7113	0.4066	n/a	0.2242	0.0210	n/a
2	0.3440	0.2306	n/a	0.0385	0.0106	n/a
3	0.2404	0.1764	n/a	0.0248	0.0070	n/a
4	0.1904	0.1467	n/a	0.0180	0.0069	n/a
5	0.1634	0.1321	n/a	0.0144	0.0065	n/a
6	0.1445	0.1163	n/a	0.0122	0.0036	n/a
7	0.1302	0.1068	n/a	0.0108	0.0047	n/a
8	0.1148	0.0915	n/a	0.0087	0.0037	n/a
9	0.0927	0.0635	n/a	0.0082	0.0029	n/a
10	0.0890	0.0605	0.1303	0.0076	0.0027	0.0886

Appendix V

Pickup Zone Fine Tuning Model Results

	Random Weights			Pre-trained Weights		
Epoch	Train Acc	Validation Acc	Test Acc	Train Acc	Validation Acc	Test Acc
1	3.8499%	4.8428%	n/a	36.4261%	59.4887%	n/a
2	4.9196%	4.8428%	n/a	64.2280%	71.4105%	n/a
3	5.1398%	4.9863%	n/a	71.1968%	76.0463%	n/a
4	6.1845%	6.3547%	n/a	74.6379%	79.6075%	n/a
5	7.0072%	7.0072%	n/a	77.3363%	82.0973%	n/a
6	7.8082%	8.2838%	n/a	79.2888%	83.7027%	n/a
7	8.3689%	8.6209%	n/a	80.6588%	85.6018%	n/a
8	8.6843%	8.9213%	n/a	82.0506%	87.0236%	n/a
9	9.1950%	9.9259%	n/a	83.0285%	88.2818%	n/a
10	9.8775%	10.4466%	9.7520%	84.1649%	89.1563%	89.0799%

Appendix VI

Dropoff Zone Fine Tuning Model Results

	Random Weights			Pre-trained Weights		
Epoch	Train Acc	Validation Acc	Test Acc	Train Acc	Validation Acc	Test Acc
1	9.3602%	11.7949%	n/a	39.4083%	63.6473%	n/a
2	11.7457%	11.7998%	n/a	67.6457%	75.0551%	n/a
3	11.7499%	11.7949%	n/a	74.1639%	80.2750%	n/a

4	11.7512%	11.7903%	n/a	77.3413%	82.8282%	n/a
5	11.7783%	11.9551%	n/a	79.1135%	84.3101%	n/a
6	12.0386%	12.1053%	n/a	80.4502%	85.5083%	n/a
7	12.1254%	12.1187%	n/a	81.2396%	86.0190%	n/a
8	12.1854%	12.1420%	n/a	81.5717%	86.3661%	n/a
9	12.2272%	12.2088%	n/a	82.2125%	86.5229%	n/a
10	12.4408%	12.5492%	12.5855%	82.6831%	87.5108%	87.3177%

7 - References

- [1] M. M. Alam, L. Torgo, and A. Bifet, “A Survey on Spatio-temporal Data Analytics Systems,” *ACM Comput. Surv.*, vol. 54, no. 10s, p. 219:1-219:38, Nov. 2022, doi: 10.1145/3507904.
- [2] G. Atluri, A. Karpatne, and V. Kumar, “Spatio-Temporal Data Mining: A Survey of Problems and Methods,” *ACM Comput. Surv.*, vol. 51, no. 4, p. 83:1-83:41, Aug. 2018, doi: 10.1145/3161602.
- [3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” May 24, 2019, *arXiv*: arXiv:1810.04805. doi: 10.48550/arXiv.1810.04805.
- [4] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving Language Understanding by Generative Pre-Training”.
- [5] A. Hagen, “Turing-NLG: A 17-billion-parameter language model by Microsoft,” Microsoft Research. Accessed: Mar. 06, 2025. [Online]. Available: <https://www.microsoft.com/en-us/research/blog/turing-nlg-a-17-billion-parameter-language-model-by-microsoft/>
- [6] C. Yang *et al.*, “Big Earth data analytics: a survey,” *Big Earth Data*, vol. 3, no. 2, pp. 83–107, Apr. 2019, doi: 10.1080/20964471.2019.1611175.
- [7] D. Guo *et al.*, “DeepSeek-Coder: When the Large Language Model Meets Programming -- The Rise of Code Intelligence,” Jan. 26, 2024, *arXiv*: arXiv:2401.14196. doi: 10.48550/arXiv.2401.14196.
- [8] M. Moor *et al.*, “Foundation models for generalist medical artificial intelligence,” *Nature*, vol. 616, no. 7956, pp. 259–265, Apr. 2023, doi: 10.1038/s41586-023-05881-4.
- [9] Y. Qin *et al.*, “Tool Learning with Foundation Models,” *ACM Comput. Surv.*, vol. 57, no. 4, pp. 1–40, Apr. 2025, doi: 10.1145/3704435.
- [10] B. Zoph *et al.*, “Rethinking Pre-training and Self-training,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2020, pp. 3833–3845. Accessed: Mar. 23, 2025. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/27e9661e033a73a6ad8cefcde965c54d-Abstract.html>
- [11] D. Hendrycks, K. Lee, and M. Mazeika, “Using Pre-Training Can Improve Model Robustness and Uncertainty,” in *Proceedings of the 36th International Conference on Machine Learning*, PMLR, May 2019, pp. 2712–2721. Accessed: Mar. 23, 2025. [Online]. Available: <https://proceedings.mlr.press/v97/hendrycks19a.html>

- [12] L. Semenova, C. Rudin, and R. Parr, “On the Existence of Simpler Machine Learning Models,” in *2022 ACM Conference on Fairness, Accountability, and Transparency*, Seoul Republic of Korea: ACM, Jun. 2022, pp. 1827–1858. doi: 10.1145/3531146.3533232.
- [13] T. Shaikhina, D. Lowe, S. Daga, D. Briggs, R. Higgins, and N. Khovanova, “Machine Learning for Predictive Modelling based on Small Data in Biomedical Engineering,” *IFAC-PapersOnLine*, vol. 48, no. 20, pp. 469–474, Jan. 2015, doi: 10.1016/j.ifacol.2015.10.185.
- [14] Z. Liu, X. Yu, Y. Fang, and X. Zhang, “GraphPrompt: Unifying Pre-Training and Downstream Tasks for Graph Neural Networks,” in *Proceedings of the ACM Web Conference 2023*, Austin TX USA: ACM, Apr. 2023, pp. 417–428. doi: 10.1145/3543507.3583386.
- [15] R. Mahmood *et al.*, “How Much More Data Do I Need? Estimating Requirements for Downstream Tasks,” presented at the Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 275–284. Accessed: Mar. 23, 2025. [Online]. Available:
https://openaccess.thecvf.com/content/CVPR2022/html/Mahmood_How_Much_More_Data_Do_I_Need_Estimating_Requirements_for_CVPR_2022_paper.html
- [16] “Design of Data Masking Architecture and Analysis of Data Masking Techniques for Testing,” *International journal of engineering science and technology*, vol. 3, no. 6, pp. 5150–5159, Jun. 2011, Accessed: Mar. 23, 2025. [Online]. Available:
<https://scispace.com/papers/design-of-data-masking-architecture-and-analysis-of-data-59ag022ig3>
- [17] K. Muralidhar and R. Sarathy, “Data Shuffling—A New Masking Approach for Numerical Data,” *Management Science*, vol. 52, no. 5, pp. 658–670, 2006, doi: 10.1287/mnsc.1050.0503.
- [18] X. Wu, Y. Dong, C. Huang, J. Xu, D. Wang, and N. V. Chawla, “UAPD: Predicting Urban Anomalies from Spatial-Temporal Data,” in *Machine Learning and Knowledge Discovery in Databases*, M. Ceci, J. Hollmén, L. Todorovski, C. Vens, and S. Džeroski, Eds., Cham: Springer International Publishing, 2017, pp. 622–638. doi: 10.1007/978-3-319-71246-8_38.
- [19] T. G. Dietterich, “Machine Learning for Sequential Data: A Review,” in *Structural, Syntactic, and Statistical Pattern Recognition*, T. Caelli, A. Amin, R. P. W. Duin, D. de Ridder, and M. Kamel, Eds., Berlin, Heidelberg: Springer, 2002, pp. 15–30. doi: 10.1007/3-540-70659-3_2.
- [20] “Traffic Census Program | Caltrans.” Accessed: Mar. 02, 2025. [Online]. Available:
<https://dot.ca.gov/programs/traffic-operations/census>
- [21] “Desheng Zhang, Rutgers University.” Accessed: Mar. 02, 2025. [Online]. Available:
<https://people.cs.rutgers.edu/~dz220/data.html>

- [22] “NYC DOT - Data Feeds.” Accessed: Mar. 02, 2025. [Online]. Available: <https://www.nyc.gov/html/dot/html/about/datafeeds.shtml>
- [23] “FOILing NYC’s Taxi Trip Data – Chris Whong.” Accessed: Mar. 02, 2025. [Online]. Available: https://chriswhong.com/open-data/foil_nyc_taxi/
- [24] “Google Maps,” Google Maps. Accessed: Mar. 05, 2025. [Online]. Available: https://www.google.com/maps/@40.7615262,-73.9416305,12.54z?entry=ttu&g_ep=EgoyMDI1MDMwMi4wIKXMDSOASAFQAw%3D%3D
- [25] A. Vaswani *et al.*, “Attention Is All You Need,” 2017, *arXiv*. doi: 10.48550/ARXIV.1706.03762.